

COMPILER DESIGN

Compiler design refers to the process of creating a special software tool called a compiler, which helps in translating and converting high-level programming code written by humans into a format that can be understood and executed by a computer. It involves various steps like analyzing the code's structure, checking for errors, optimizing the code for better performance, and generating the final executable program. In simpler terms, compiler design is all about building a translator that turns human-readable code into instructions that a computer can follow.

A compiler is a special tool that helps translate code written in a high-level programming language (like Python or Java) into a language that the computer's processor can understand, which is called machine language.

The high-level language is what developers use to write code because it's easier for humans to understand. The machine language, on the other hand, is what the computer actually understands and can execute.

When you write and run a program, the compiler checks the code for errors and makes sure it follows the rules of the programming language. Its main job is to convert the code written in one language into another language without changing the program's meaning.

The program execution happens in two parts :-

1. First, the compiler takes the code you wrote (source program) and translates it into a lower-level language called the object program. This translation makes the code easier for the computer to work with.
2. Then, the object program is further translated into the target program using a tool called an assembler. The target program is the final version that the computer can directly understand and execute.

In simpler terms, a compiler is like a language translator that helps turn human-readable code into a language that the computer can understand and run. It makes sure the code is correct, and then it converts it into a form that the computer can execute.

PHASES OF COMPILER

- **Lexical Analysis:** This is the first phase of the compiler. It reads the source code character by character and groups them into meaningful units called lexemes. For example, it recognizes keywords, identifiers, numbers, and symbols in the code. These lexemes are then converted into tokens, which are representations of these meaningful units.
- **Syntax Analysis:** In this phase, the compiler takes the tokens generated in the lexical analysis phase and checks if they form valid expressions according to the rules of the programming language's syntax. It constructs a parse tree, which represents the structure and relationships between the components of the code. The parser ensures that the code is syntactically correct.
- **Semantic Analysis:** The semantic analysis phase checks whether the parse tree, generated in the syntax analysis phase, follows the rules of the programming language in terms of meaning and context. It verifies the types of identifiers, expressions, and statements. This phase also performs tasks like type checking and symbol table management to ensure that the code has meaningful semantics.
- **Intermediate Code Generation:** Here, the compiler converts the source code into an intermediate representation that is closer to the machine language but still independent of the target machine. This intermediate code should be easy to translate into the final machine code. It

helps in further analysis and optimizations before generating the actual target code.

- **Code Optimization:** This phase is optional but aims to improve the intermediate code generated in the previous phase. It analyzes the code for opportunities to make it more efficient and optimized in terms of execution speed and memory usage. This can involve eliminating redundant code, rearranging the sequence of statements, or applying mathematical transformations to simplify expressions.
- **Code Generation:** The final phase of the compilation process is code generation. It takes the optimized intermediate code and translates it into the specific machine language of the target computer. The code generator generates the instructions that the computer's processor can understand and execute. This phase produces the final executable program that can be run on the target machine.

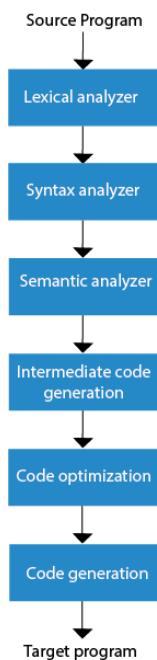
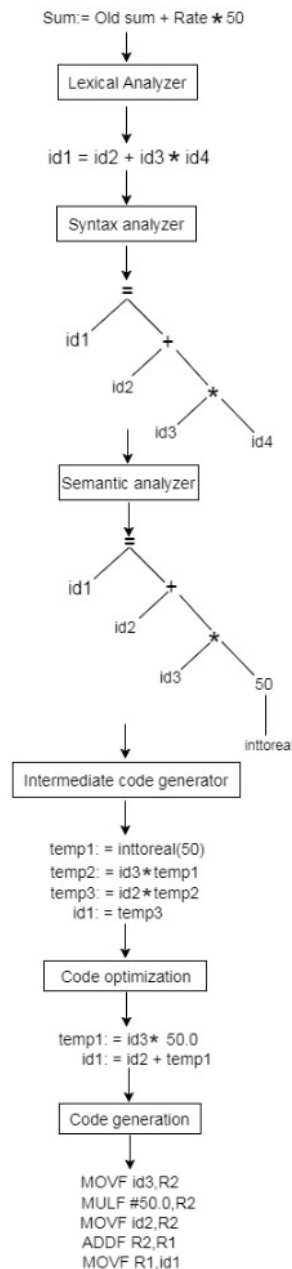


Fig: phases of compiler

In simpler terms, the compiler goes through several steps: analyzing the code's structure and meaning, generating an intermediate representation, optimizing the code if needed, and finally translating it into the machine language of the target computer. Each phase plays a specific role in converting human-readable code into executable machine code.

Example:



LEXICAL ANALYSIS

Lexical analysis refers to the process of analyzing the source code of a program to identify and extract meaningful units called lexemes. It is the first phase of the compilation process.

During lexical analysis, the compiler reads the source code character by character and groups them into lexemes. Lexemes represent the smallest meaningful units in a programming language, such as keywords, identifiers, operators, constants, and punctuation symbols.

The lexical analyzer, also known as the lexer or scanner, is responsible for recognizing these lexemes by applying predefined rules and patterns specific to the programming language. It scans the source code, identifies lexemes based on these rules, and generates a sequence of tokens.

Tokens are abstract representations of lexemes and carry more meaning in the context of the programming language. They provide a higher-level view of the code, categorizing the lexemes into different types like keywords, identifiers, operators, or literals.

The purpose of lexical analysis is to transform the continuous stream of characters in the source code into a structured sequence of tokens. This structured representation makes it easier for subsequent compiler phases, such as syntax analysis and semantic analysis, to analyze and understand the code's structure and meaning.

In simpler terms, lexical analysis is like breaking down the source code into meaningful chunks and assigning labels to these chunks. It helps the compiler recognize and categorize different parts of the code, which is important for understanding and processing the code correctly.

Lexical analysis is the first phase of the compilation process in which the compiler reads the source code character by character. Its main job is to identify and extract meaningful units called lexemes from the code.

Think of the source code as a long string of characters. The lexical analyzer breaks down this

string into smaller chunks, such as keywords, identifiers, numbers, operators, and punctuation symbols. These smaller chunks are the lexemes.

For example, let's take the following line of code:

```
x = 10 + y
```

During the lexical analysis phase, the lexical analyzer will recognize and extract the following lexemes :-

- The identifier 'x'
- The assignment operator '='
- The number '10'
- The addition operator '+'
- The identifier 'y'

Once the lexemes are identified, they are converted into tokens, which are abstract representations of these lexemes. These tokens will be used in later stages of the compiler to analyze the code's syntax and semantics.

After the lexical analyzer identifies the lexemes (meaningful units) from the source code, the next step is to convert them into tokens. Tokens are abstract representations of these lexemes and carry more meaning in the context of the programming language.

To convert lexemes into tokens, the compiler uses a set of predefined rules and patterns specific to the programming language. These rules define the different categories or types of tokens that can be encountered in the code. Each token has a specific meaning and role in the language's syntax.

For example, let's consider the following line of code in the Python programming language:

```
x = 10 + y
```

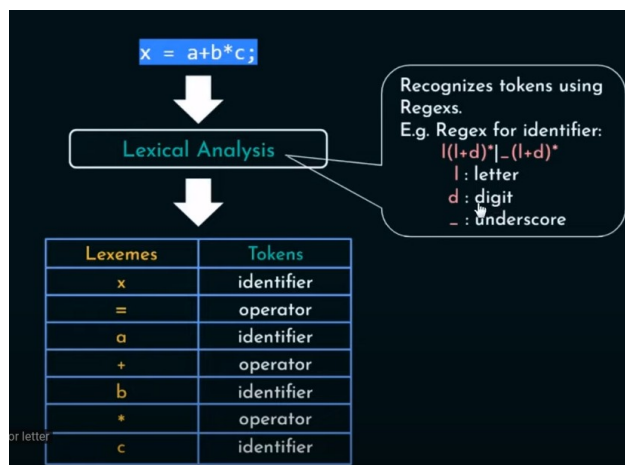
The lexical analyzer has identified the following lexemes:

- The identifier lexeme: "x" and "y"
- The assignment operator lexeme: "="
- The number lexeme: "10"
- The addition operator lexeme: "+"

Now, let's see how these lexemes are converted into tokens:

- The identifier lexeme "x" becomes an identifier token.
- The assignment operator lexeme "=" becomes an assignment token.
- The number lexeme "10" becomes a numeric constant token.
- The addition operator lexeme "+" becomes an addition operator token.

These tokens provide a higher-level representation of the code. They carry information about the type of the lexeme and its role in the programming language. The tokens are then used in subsequent phases of the compiler, such as syntax analysis and semantic analysis, to understand the structure and meaning of the code.



In simpler terms, converting lexemes into tokens is like giving special names or labels to different parts of the code. These names or labels help us understand the role and meaning of each part. It's like putting a tag on each word or symbol to say what it represents in the programming language.

SYNTAX ANALYSIS

Syntax analysis, also known as parsing, is the second phase of the compilation process after lexical analysis. It focuses on analyzing the structure of the source code based on the grammar rules of the programming language.

Syntax analysis checks whether the sequence of tokens generated from the lexical analysis phase conforms to the rules specified by the language's syntax. It ensures that the code follows the correct arrangement and combination of tokens to form valid expressions, statements, and program structures.

Syntax analysis typically constructs a parse tree or syntax tree, which is a hierarchical representation of the code's structure. The parse tree shows how the different parts of the code relate to each other based on the language's syntax rules.

Here's a simplified example in the C programming language to illustrate syntax analysis:

Consider the code:

```
int x = 10;
```

During syntax analysis, the parser checks the tokens generated from lexical analysis against the grammar rules of the C language. It verifies that the tokens form a valid declaration statement.

In this case, the syntax analysis will construct a parse tree like this:

```
declaration
 /  |  \
int =  ;
 |  |  |
x  10 null
```

The parse tree reflects the structure of the declaration statement, showing that it starts with the keyword "int," followed by an identifier "x," the assignment operator "=", and a numeric constant "10." The tree also indicates that the semicolon ";" marks the end of the statement.

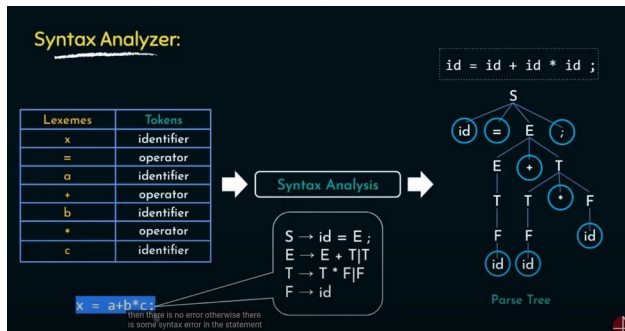
Syntax analysis detects syntax errors if the tokens cannot be arranged according to the language's

grammar rules. For example, consider the following invalid code:

```
int x = ;
```

The syntax analysis will detect an error because the assignment operator "=" expects an expression on the right side, but in this case, it encounters a semicolon without a valid expression.

Example :-



Syntax analysis is crucial because it ensures that the code follows the correct structure and grammar of the programming language. By constructing the parse tree, it provides a structural representation of the code that can be used for subsequent analysis, optimization, and code generation phases of the compiler.

SEMANTIC ANALYZER

Semantic analysis is the phase of the compilation process that follows syntax analysis. It focuses on understanding the meaning and correctness of the code beyond its structure. Semantic analysis checks for semantic errors and ensures that the code makes sense according to the rules and constraints of the programming language.

Here's a brief explanation of semantic analysis and an example to illustrate its workings:

Semantic analysis involves performing various tasks such as :-

1. **Type Checking:** The compiler verifies that the operations and expressions in the code are applied to compatible types. For example, adding a number to a string would be a type mismatch and flagged as an error.

2. **Scope Analysis:** The compiler checks the scope rules to ensure that variables and identifiers are declared and used correctly within their respective scopes. It ensures that variables are declared before use and are accessible where they are referenced.

3. **Declaration Analysis:** The compiler verifies that variables, functions, and other symbols are properly declared and used according to the language's rules. It checks for issues like re-declaration, multiple definitions, or referencing an undeclared symbol.

4. **Semantic Constraints:** The compiler enforces additional semantic constraints specific to the programming language. For example, ensuring that a function call has the correct number and type of arguments.

Semantic analysis ensures that the code is semantically valid based on the information provided by the parse tree generated during the syntax analysis phase. It performs various checks and validations to ensure that the code makes sense and follows the rules and constraints of the programming language. Let's explore some of the tasks performed during semantic analysis in more detail :-

1. **Type Checking:** Semantic analysis verifies that the operations and expressions in the code are applied to compatible types. It checks that variables, constants, and expressions are used in a manner consistent with their declared types.

For example, consider the following code snippet in C:

```
c
Copy code

int x = 10;
char y = 'a';
int z = x + y;
```

During semantic analysis, the compiler would perform type checking and detect an error in the line `int z = x + y;`. This error arises because the addition operation is not allowed between an

integer (x) and a character (y). The compiler would report a type mismatch error.

2. **Scope Analysis:** Semantic analysis checks that variables and identifiers are used within their proper scopes. It ensures that variables are declared before use and are accessible where they are referenced. The compiler maintains a symbol table to track the information about variables and their scopes.

For example, consider the following code snippet in Python:

```
python
x = 10

def my_function():
    print(x)

my_function()
```

During semantic analysis, the compiler would ensure that the reference to x inside the function my_function is valid. It checks the symbol table to verify that x is accessible from the function's scope.

3. **Declaration Analysis:** Semantic analysis verifies that variables, functions, and other symbols are correctly declared and used according to the language's rules. It checks for issues such as re-declaration, multiple definitions, or referencing an undeclared symbol.

For example, consider the following code snippet in Java:

```
java
int x = 10;
int x = 20;
```

During semantic analysis, the compiler would detect an error because the variable x is re-declared. This violates the language's rule that a variable should only be declared once in the same scope.

4. **Semantic Constraints :-** Semantic analysis enforces additional semantic constraints specific to the programming language. These constraints can include rules

```
java
class Rectangle {
    int width;
    int height;
}

Rectangle r = new Rectangle();
r.area();
```

Semantic analysis will identify an error in the line r.area() because the area() method is not defined within the Rectangle class. It ensures that the code follows the language's rules regarding method calls and member accessibility.

By performing these checks, semantic analysis verifies the correctness and meaningfulness of the code. It helps in generating appropriate error messages and enables subsequent compiler phases like optimization and code generation to work with reliable and valid code.

INTERMEDIATE CODE GENERATOR

The intermediate code generator is a phase in the compilation process that converts the source code into an intermediate representation that is closer to the target machine code. It serves as a bridge between the high-level programming language and the final machine code.

The intermediate code generator takes the input from previous phases, such as syntax and semantic analysis, and generates an intermediate code representation. This intermediate code is designed to be easier to analyze, optimize, and translate into the target machine code.

The process of intermediate code generation involves several steps:

1. **Mapping Expressions and Statements:** The intermediate code generator maps high-level language constructs, such as expressions and statements, into equivalent representations in the intermediate code. This mapping captures the essence of the original code in a more abstract form.
2. **Handling Control Flow:** The generator handles control flow constructs, such as conditionals (if-else) and loops (while,

for), and represents them in the intermediate code. This may involve generating intermediate code instructions to handle branches, jumps, and loops.

3. **Managing Data and Variables:** The generator assigns memory locations and manages the storage of variables and data in the intermediate code. It keeps track of data types, declarations, and references, ensuring consistent and efficient memory usage.
4. **Handling Function Calls:** The generator deals with function calls, including passing arguments, managing return values, and maintaining the execution context during function invocations.
5. **Generating Code Annotations:** The intermediate code generator may include annotations or additional information to aid subsequent optimization or code generation phases. These annotations can provide hints or constraints for further analysis and transformation.

Example: Let's consider the following code snippet in C:

```
int x = 5;  
int y = 10;  
int z = x + y;
```

During intermediate code generation, the generator can produce an intermediate representation like three-address code :-

```
t1 = 5  
t2 = 10  
t3 = t1 + t2
```

In this example, the intermediate code generator translates the variable assignments and the addition operation into three-address code. It assigns the value 5 to temporary variable t1, the value 10 to t2, and performs the addition of t1 and t2, storing the result in t3.

The generated intermediate code is closer to the machine code but still independent of the specific target machine architecture. It provides a more

manageable representation for subsequent optimization and code generation phases.

The purpose of the intermediate code generator is to facilitate further analysis and transformations on the code before generating the final machine code. By using an intermediate representation, the compiler can apply various optimizations and target-specific translations in a more structured and efficient manner.

CODE OPTIMIZATION

Code optimization is a phase in the compilation process that aims to improve the efficiency, speed, and overall performance of the generated code. It focuses on transforming the code to produce an optimized version while preserving its original functionality.

The code optimization phase analyzes the code to identify opportunities for improvement and applies various techniques to make the code more efficient. These techniques can include removing redundant operations, reordering instructions, simplifying expressions, and minimizing memory usage.

Here's an overview of how code optimization works:

1. **Analysis:** The optimizer analyzes the code to identify patterns, dependencies, and potential areas for improvement. It examines the control flow, data dependencies, and resource usage within the code.
2. **Identifying Optimization Opportunities:** Based on the analysis, the optimizer identifies specific areas where the code can be optimized. This can include eliminating redundant computations, reducing memory access, and improving loop structures.
3. **Transformation:** The optimizer applies various transformation techniques to improve the code. This can involve reordering instructions, eliminating

unnecessary branches or loops, and simplifying complex expressions.

4. **Optimization Techniques:** The optimizer employs a range of techniques such as constant folding, common subexpression elimination, loop unrolling, register allocation, and instruction scheduling. These techniques aim to eliminate inefficiencies, reduce overhead, and exploit parallelism.
5. **Trade-offs:** During optimization, the compiler needs to strike a balance between code efficiency and other factors such as code size, compilation time, and maintainability. Some optimizations may result in increased code size or longer compilation time, so the compiler must consider these trade-offs.

DIFFERENT TECHNIQUES OF OPTIMIZATION

Here's a more detailed explanation of various optimization techniques commonly applied during the code optimization phase :-

1. **Constant Folding** :- This optimization technique involves evaluating expressions with constant values at compile-time instead of at runtime. It replaces the expression with the computed result. For example:

```
int x = 5;
int y = x + 3;
```

During constant folding, the expression $x + 3$ is computed as $5 + 3$ (which is 8) at compile-time. The code is then transformed to:

```
int x = 5;
int y = 8;
```

- **Common Sub expression Elimination:** This technique identifies and eliminates redundant computations by storing the result of a sub expression in a temporary variable. The temporary variable is then reused wherever the sub expression appears. For example:

```
int x = 5;
int y = x + 3;
int z = x * y;
```

During common sub expression elimination, the optimizer recognizes that x is used twice and stores its value in a temporary variable. The code is transformed to:

```
int x = 5;
int temp = x;
int y = temp + 3;
int z = temp * y;
```

Strength Reduction :- This technique aims to replace expensive operations with cheaper ones. For example, multiplication can be replaced with repeated addition or division with bit shifting. Here's an example:

```
int x = 10;
int y = x * 4;
```

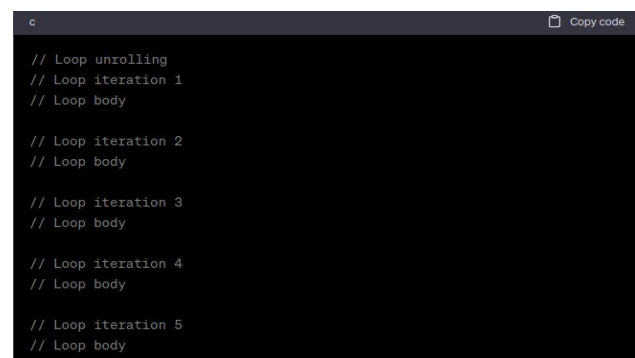
During strength reduction, the optimizer replaces the multiplication by 4 with a series of additions:

```
int x = 10;
int y = x + x + x + x;
```

Loop Unrolling: Loop unrolling reduces loop overhead by replicating loop iterations. Instead of executing the loop for each iteration, it performs multiple iterations in a single pass. This can improve performance by reducing loop control instructions and improving instruction pipelining. Here's an example:

```
for (int i = 0; i < 5; i++) {
    // Loop body
}
```

During loop unrolling, the optimizer replicates the loop body multiple times:



```
c Copy code
// Loop unrolling
// Loop iteration 1
// Loop body

// Loop iteration 2
// Loop body

// Loop iteration 3
// Loop body

// Loop iteration 4
// Loop body

// Loop iteration 5
// Loop body
```


These are just a few examples of the many optimization techniques used in the code optimization phase. Other techniques include register allocation, instruction scheduling, function inlining, dead code elimination, and more. Each technique aims to improve the code's performance, reduce overhead, and make more efficient use of system resources. The specific optimization techniques applied depend on the compiler and the target platform.

TARGET CODE GENERATOR

The target code generator is the final phase of the compilation process. It takes the optimized intermediate code representation and translates it into the specific machine code instructions that can be executed directly on the target hardware or platform.

The target code generator is responsible for generating efficient and executable code that closely corresponds to the architecture and instruction set of the target machine. It considers the specific features and constraints of the target platform to produce optimized machine code.

Here's a detailed explanation of the target code generator and examples to illustrate its workings:

1. **Instruction Selection:** The target code generator selects appropriate machine instructions based on the intermediate code instructions. It maps each operation and expression in the intermediate code to a corresponding sequence of machine instructions. For example:

Intermediate Code:

`t1 = a + b`

Target Machine Code (x86 assembly):

`ADD eax, ebx`

In this example, the target code generator selects the ADD instruction in x86 assembly to perform the addition operation.

2. **Register Allocation:** The target code generator assigns intermediate variables and values to specific registers or memory

locations in the target machine. It considers the availability and limitations of registers in the target architecture. For example:

Intermediate Code:

`t1 = a + b`

`t2 = c * d`

Target Machine Code (ARM assembly):

`ADD r1, r2, r3`

`MUL r4, r5, r6`

3. **Memory Access and Addressing Modes:**

The target code generator handles memory access and determines the appropriate addressing modes for loading or storing data in memory. It optimizes memory access patterns to minimize latency and maximize cache efficiency.

For example:

Intermediate Code:

`x = y + z`

Target Machine Code (MIPS assembly):

```
lw $t0, y    ; Load y into register $t0
lw $t1, z    ; Load z into register $t1
add $s0, $t0, $t1 ; Add y and z,
store result in $s0
sw $s0, x    ; Store result in x
```

In this example, the target code generator uses the lw instruction to load values from memory into registers and the sw instruction to store the result back into memory.

4. **Optimization and Target-Specific Features:**

The target code generator may perform additional optimizations specific to the target machine architecture. It can exploit pipeline features, vectorization, or other architectural characteristics for improved performance. The generated code may also include target-specific instructions or extensions. For example:

Intermediate Code:

```
x = a * b
```

Target Machine Code (SIMD instruction for vectorized multiplication):

```
VMULPS xmm0, xmm1, xmm2
```

In this example, the target code generator recognizes the opportunity for vectorization and generates SIMD (Single Instruction, Multiple Data) instructions to perform a vectorized multiplication.

The target code generator tailors the generated machine code to the specific target machine architecture, optimizing it for efficient execution. It leverages the features and capabilities of the target platform to produce code that takes advantage of the hardware resources and maximizes performance.

It's important to note that the target code generator is highly dependent on the target architecture and can vary significantly between different machines or platforms.

CHAPTER TWO

LEXICAL ANALYSIS

Lexical analysis, also known as scanning or tokenization, is an important phase in the compilation process. Its purpose is to break down the source code into smaller, meaningful units called lexemes or tokens. These tokens serve as the building blocks for further analysis and processing by the compiler.

During lexical analysis, the source code is read character by character. The lexical analyzer, also called a lexer or scanner, scans the code and identifies different lexemes based on predefined rules and patterns specific to the programming language.

Here's an overview of how lexical analysis works:

1. **Scanning:** The lexical analyzer reads the source code character by character, ignoring whitespaces and comments that do not affect the meaning of the code.
2. **Lexeme Recognition:** As the lexical analyzer scans the source code, it identifies and recognizes lexemes based on the language's grammar rules and lexical conventions. Lexemes can include keywords, identifiers, operators, literals, and punctuation symbols.
3. **Token Generation:** Once a lexeme is recognized, it is transformed into a token. Tokens are categorized based on their role and type in the programming language. For example, a token can represent a keyword, an identifier, a numeric constant, or an operator.
4. **Token Output:** The lexical analyzer outputs the generated tokens to the subsequent phases of the compiler for further analysis and processing. The tokens provide a higher-level representation of the code, making it easier to analyze its structure and meaning.

Let's consider an example in the C programming language :-

```
int x = 42;
```

During lexical analysis, the following tokens would be generated:

- Keyword token: int
- Identifier token: x
- Operator token: =
- Numeric constant token: 42
- Punctuation token: ;

These tokens represent the meaningful components of the code, such as the keyword int, the variable name x, the assignment operator =, the numeric constant 42, and the semicolon ;.

The lexical analysis phase is crucial as it provides a foundation for subsequent phases like syntax analysis and semantic analysis. It transforms the continuous stream of characters into a structured sequence of tokens, enabling the compiler to understand and process the code effectively.

COUNTING THE NUMBER TOKENS IN A CODE SEGMENT

Counting the number of tokens in a code segment involves identifying and categorizing the individual tokens present in the code. Tokens represent the smallest meaningful units of code, such as keywords, identifiers, operators, literals, and punctuation symbols. Here's a detailed explanation of the techniques for counting tokens and an example to illustrate the process :-

Techniques for Counting Tokens :-

1. **Manual Counting:** This technique involves visually inspecting the code segment and identifying each token by following the language's syntax rules and lexical conventions. Tokens are counted manually, and a tally is kept for each token type encountered.
2. **Lexical Analysis Tools:** Lexical analysis tools, such as Lex, Flex, or ANTLR, automate the process of tokenization.

These tools generate lexical analyzers based on predefined rules and patterns specified using regular expressions or other formal grammars. The generated lexer scans the code segment and produces tokens, making it easy to count and categorize them.

Example: Let's consider a simple code segment in the C programming language:

```
c Copy code
int main() {
    int x = 5;
    int y = 10;
    int sum = x + y;
    return sum;
}
```

Using manual counting, we can identify and count the tokens :-

1. Keywords:

- int: 4
- main: 1
- return: 1

2. Identifiers:

- x: 3
- y: 2
- sum: 2

3. Punctuation:

- (: 1
-): 1
- {: 1
- }: 1
- :: 3

4. Operators:

- =: 3
- +: 1

5. Numeric Literals:

- 5: 1
- 10: 1

By tallying the counts, we can determine that the code segment contains :-

- 7 keyword tokens
- 7 identifier tokens
- 6 punctuation tokens
- 4 operator tokens
- 2 numeric literal tokens

In total, there are 26 tokens in the given code segment.

Using lexical analysis tools, the token counting process becomes automated. The lexical analyzer generated by the tool would handle the scanning and tokenization, providing an accurate count of tokens based on the specified rules.

Counting tokens in a code segment is essential for understanding the code's structure, identifying its components, and performing subsequent phases of compilation. It helps in analyzing the code's syntax, detecting errors, and generating meaningful representations for further processing by the compiler.

Example :-

Lexical Analyzer-Tokenization:

Tokens:

1. **Keyword:** int, return
2. **Identifier:** main, x, a, b, c, printf
3. **Punctuator:** (,), {, }, ,, ;, }
4. **Operator:** =, +, *
5. **Constant:** 2, 3, 5, 0
6. **Literal:** "The value of x is %d"

Count: 39

```
int main()
{
    int x,a=2,b=3,c=5;
    x = a+b*c;
    printf("The value of x is %d",x);
    return 0;
}
```

CHAPTER THREE

SYNTAX ANALYSIS

The second phase of compiler design is syntax analysis, also known as parsing. The primary goal of syntax analysis is to ensure that the source code follows the rules and structure specified by the language's grammar. It involves analyzing the sequence of tokens generated by the lexical analysis phase and constructing a parse tree or syntax tree that represents the syntactic structure of the program.

Syntax analysis verifies the correctness of the program's syntax by checking if the sequence of tokens can be derived from the grammar of the programming language. This process involves applying a set of production rules defined by a context-free grammar (CFG) or a similar formalism. The CFG describes the syntax rules of the language and determines how different language constructs can be combined and nested.

During syntax analysis, the following steps are typically performed:

1. **Tokenization** :- The input source code is divided into tokens using the rules specified by the lexical analyzer. Each token represents a meaningful unit of the programming language, such as identifiers, keywords, operators, and literals.
2. **Parsing**: The tokens are processed and organized hierarchically to create a parse tree or syntax tree. The parse tree represents the syntactic structure of the program and shows how the various language constructs are related to each other. There are different parsing techniques, such as top-down parsing (e.g., recursive descent parsing) and bottom-up parsing (e.g., LR parsing), which systematically apply the grammar rules to construct the parse tree.
3. **Error Handling**: If any syntax errors are encountered during parsing, error

messages are generated to inform the programmer about the specific issues in the code. These errors may include missing semicolons, mismatched parentheses, or incorrect usage of language constructs.

The output of the syntax analysis phase is either a parse tree or an abstract syntax tree (AST). An AST simplifies and abstracts the parse tree by removing unnecessary details and focuses on the essential elements of the program's syntax. The resulting parse tree or AST is then used in subsequent phases of the compiler, such as semantic analysis and code generation.

Syntax analysis is a crucial step in the compilation process as it ensures that the source code adheres to the language's grammar and can be further processed and transformed into executable code.

THE ROLE OF PARSER

The parser plays a crucial role in the syntax analysis phase of the compiler. Its main task is to take the sequence of tokens generated by the lexical analyzer and determine whether this sequence adheres to the grammar rules of the source language. The grammar used by the parser is typically a Context-Free Grammar (CFG), which provides a set of production rules for generating valid program structures.

The parser's primary objective is to ensure that the input program is well-formed and syntactically correct. It accomplishes this by constructing a parse tree, which represents the hierarchical structure of the program based on the grammar rules. The parse tree serves as a visual representation of how the different language constructs are nested within each other.

There are two main types of parsers used in compilers: top-down parsers and bottom-up parsers.

1. **Top-Down Parsing**: In top-down parsing, the parser starts from the root of the parse tree and works its way down to the

leaves. It begins with the start symbol of the grammar and recursively applies production rules to generate the parse tree. The most common top-down parsing method is Recursive Descent Parsing, where each non-terminal in the grammar is associated with a separate procedure or function.

2. **Bottom-Up Parsing:** In bottom-up parsing, the parser starts from the input tokens and gradually builds the parse tree by reducing the tokens according to the grammar rules. Bottom-up parsing algorithms, such as LR (Left-to-Right, Rightmost derivation) or LALR (Look-Ahead LR) parsing, are commonly used to construct the parse tree.

Regardless of the parsing method used, the parser scans the input program from left to right, examining one symbol (token or non-terminal) at a time. If the parser encounters any syntax errors, such as an invalid sequence of tokens or a violation of the grammar rules, it reports these errors to the programmer in a meaningful and understandable way. Error recovery mechanisms may also be implemented in the parser to handle common syntax errors and allow the compilation process to continue.

Once the parser constructs the parse tree successfully, it passes the parse tree or an Abstract Syntax Tree (AST) to the subsequent phases of the compiler for further processing. The parse tree or AST serves as the foundation for subsequent analysis, such as semantic analysis, optimization, and code generation.

Overall, the parser is responsible for verifying the syntactic correctness of the program by applying the grammar rules, constructing the parse tree, reporting syntax errors, and passing the resulting tree structure to the next phases of the compiler.

CONTEXT FREE GRAMMERS

In the field of compiler design, context-free grammars play a crucial role in specifying the syntactic structure of programming languages. A context-free grammar describes a set of strings or language and provides rules for composing these strings from various syntactic elements. These elements include terminals, non-terminals, start symbols, and production rules.

Terminals represent the basic symbols from which strings are formed. They correspond to the fundamental building blocks of a programming language, such as identifiers, keywords, operators, and literals. Non-terminals, on the other hand, act as placeholders that can be replaced by other terminals or non-terminals to create larger structures.

The start symbol is a designated non-terminal that indicates the entry point or top-level construct of the language's syntax. It defines the language that the grammar represents. By starting from the start symbol and applying production rules, valid programs or expressions in the language can be derived.

Production rules specify how terminals and non-terminals can be combined to form valid strings in the language. Each production rule consists of a head (left side), an arrow symbol ($\rightarrow$), and a body (right side). The head represents the construct being defined, while the body describes the components that can be used to construct valid strings.

With these components, context-free grammars provide a precise and formal way to describe the syntactic structure of programming languages. They help compilers understand and analyze the structure of programs during the syntax analysis phase, enabling the detection of syntax errors and the construction of parse trees for further processing.

- **Terminals:** Terminals are the basic symbols from which strings are formed. They represent the fundamental building blocks of a language. In the context of a compiler, terminals are often referred to as token names. Tokens are generated by the lexical analyzer and serve as input to the syntax analyzer. Each token corresponds to a specific lexeme, such as identifiers, keywords, operators, or literals.

1. **Non-Terminals:** Non-terminals, also known as syntactic variables, represent sets of strings of terminals. They act as placeholders that can be replaced by other terminals or non-terminals according to the grammar rules. Non-terminals are used to define the structure and syntax of a programming language. They are typically represented by uppercase letters or symbols.

2. **Start Symbol:** In a context-free grammar, one non-terminal is designated as the start symbol. The start symbol specifies the language that the grammar defines. It represents the top-level construct or entry point of the language's syntax. All valid programs or expressions in the language can be derived by starting from the start symbol and applying the production rules.

3. **Production Rules:** Production rules specify the different ways in which terminals and non-terminals can be combined to form strings in the language. Each production rule consists of three parts :-

a) **Head or Left Side:** The head of a production rule is a non-terminal that defines the strings generated by that rule. It represents the current construct being defined or expanded.

b) **Arrow Symbol ($\rightarrow$):** The arrow symbol separates the head from the body of the production rule. It indicates that the head can be replaced by the elements in the body.

c) **Body or Right Side:** The body of a production rule consists of zero or more terminals and non-terminals. It describes one possible way in which strings can be constructed from the non-terminal at the head. The components of the body can be terminals (tokens) or non-terminals, and they specify the structure of the language.

Production rules provide the building blocks for constructing valid strings in the language. By applying the production rules recursively, starting from the start symbol, a parser can generate the valid syntax tree or parse tree for a given program.

Overall, context-free grammars are used to specify the syntactic structure of a programming language. They define the valid combinations and arrangements of terminals and non-terminals, which are necessary for parsing and understanding the structure of a program during the syntax analysis phase of the compiler.

For example, take the following grammar and input string. $E \rightarrow E - E \mid E * E \mid a \mid b \mid c$; and input string given is “ $a - b * c$ ”.

In the given example, we have a context-free grammar with a start symbol E , terminals $-$, $*$, a , b , c , and a single non-terminal E . The grammar consists of several production rules, each with the head A (in this case, E) and alternative bodies ($\alpha_1, \alpha_2, \dots, \alpha_k$).

The production rules for this grammar are as follows :-

1. $E \rightarrow E - E$
2. $E \rightarrow E * E$
3. $E \rightarrow a$
4. $E \rightarrow b$
5. $E \rightarrow c$

The first two production rules have the same head E , and their bodies represent

alternative ways to construct expressions involving subtraction and multiplication. The third, fourth, and fifth production rules define the possible terminals that can be directly derived from E, which are the single characters a, b, and c.

To generate a parse tree for a given input string "a - b * c" using this grammar, the parser applies these production rules in a way that matches the input string. The parser starts with the start symbol E and applies the production rules to derive the input string. Here's a step-by-step breakdown :-

1. $E \rightarrow E - E$ (using the first production rule) This rule allows E to be expanded into E - E.
2. $E - E \rightarrow a - E$ (using the third production rule) The left E is expanded into a using the third production rule.
3. $a - E \rightarrow a - E * E$ (using the second production rule) The right E is expanded into E * E using the second production rule.
4. $a - E * E \rightarrow a - b * E$ (using the fourth production rule) The right E is expanded into b using the fourth production rule.
5. $a - b * E \rightarrow a - b * c$ (using the fifth production rule) The right E is expanded into c using the fifth production rule.

At this point, the parse tree has been constructed, and the input string "a - b * c" has been successfully derived using the given grammar.

The production rules in a context-free grammar define the structure of valid strings in the language it represents. By applying these rules, parsers can analyze the syntax of programs and construct parse trees that capture the hierarchical relationships between the language constructs.

DERIVATIONS

In the process of parsing, a derivation refers to a sequence of production rules that are applied to transform a given input string according to the grammar of the language. During parsing, two decisions are made: selecting the non-terminal to be replaced and determining the production rule to be used for the replacement.

To illustrate the concept of derivation, let's consider a non-terminal symbol A surrounded by grammar symbols α and β , such that the current sentential form is $\alpha A \beta$. Suppose we have a production rule $A \rightarrow \gamma$. In this case, we can write the derivation as $\alpha A \beta \Rightarrow \alpha \gamma \beta$, indicating that A has been replaced by γ .

There are two types of derivations commonly used :-

1. **Left-Most Derivation (LMD):** In a left-most derivation, the input string is scanned and replaced from left to right. At each step, the left-most non-terminal is chosen for replacement. This type of derivation ensures that the left-most variable in a production body is always replaced first, and the process continues from left to right. For example, let's consider the grammar productions :

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$

Suppose we have the input string "id + id * id". The left-most derivation for this input will be as follows:

$E \rightarrow E * E$

$E \rightarrow E + E * E$

$E \rightarrow id + E * E$

$E \rightarrow id + id * E$

$E \rightarrow id + id * id$

In each step, the left-most non-terminal (underlined) is selected for replacement.

2. **Right-Most Derivation (RMD):** In a right-most derivation, the input is scanned and replaced from right to left. At each step, the right-most non-terminal is chosen for replacement. This type of derivation ensures that the right-most non-terminal is replaced first, and the process continues from right to left. Using the same grammar productions as before, the right-most derivation for the input "id + id * id" will be as follows:

$E \rightarrow E + E$

$E \rightarrow E + E * E$

$E \rightarrow E + E * id$

$E \rightarrow E + id * id$

$E \rightarrow id + id * id$

In each step, the right-most non-terminal (underlined) is selected for replacement.

Both left-most and right-most derivations are useful for analyzing the structure of a program and understanding how the grammar rules are applied to transform the input string. They provide insights into the order of replacements and the hierarchical relationships between the grammar symbols.

DERIVATION AND PARSE TREE

In the context of parsing, a parse tree is a graphical representation of a derivation. It provides a visual depiction of how the start symbol of a grammar generates a particular string in the language. Each node in the parse tree represents a symbol (either a terminal or a non-

terminal), and the edges connecting the nodes represent the production rules used to derive the string.

Here are some key characteristics of a parse tree :

1. **Root Node:** The root of the parse tree corresponds to the start symbol of the grammar. It represents the highest-level construct in the language.
2. **Leaf Nodes:** The leaf nodes of the parse tree correspond to the terminal symbols in the input string. They represent the individual tokens or basic elements of the language.
3. **Interior Nodes:** The interior nodes of the parse tree correspond to non-terminal symbols. They represent higher-level constructs in the language and serve as the points of derivation.
4. **Children and Parent Nodes:** Each interior node has children nodes, which are labeled from left to right according to the symbols in the right-hand side of the production rule. The parent node is the non-terminal that was replaced by these symbols during the derivation.
5. **In-order Traversal:** Performing an in-order traversal of the parse tree, from left to right, results in the original input string. This traversal visits the nodes in the same order as they appear in the input.

The parse tree captures the hierarchical structure of the language and shows how the input string can be derived from the start symbol. It provides a visual representation that filters out the choice of replacement order made during the derivation process, making it easier to understand the structure and relationships within the parsed program.

Example :-

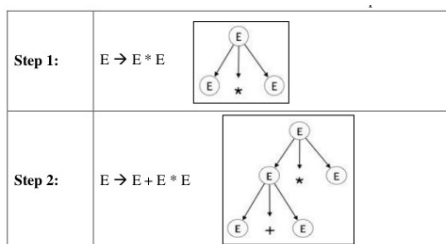
$E \rightarrow E * E$

$E \rightarrow E + E * E$

$E \rightarrow id + E * E$

$E \rightarrow id + id * E$

$E \rightarrow id + id * id$



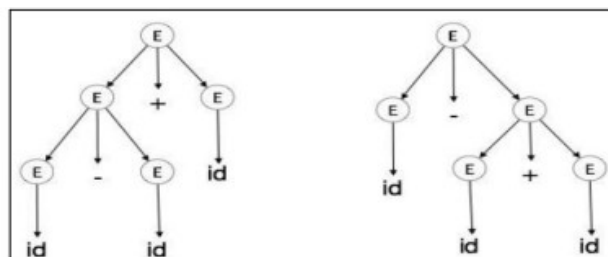
In the given example, let's consider the grammar:

$E \rightarrow E + E$

$E \rightarrow E - E$

$E \rightarrow id$

And the input string: $id - id + id$



CONTEXT FREE GRAMMERS VERSUS REGULAR GRAMMERS

Regular expressions and context-free grammars are two different notations used in language specification. Here's a brief explanation of each:

- Regular Expressions:** Regular expressions are a powerful tool for defining patterns of tokens in a string. They are used to describe regular languages, which are a subset of context-free languages. Regular expressions consist of a combination of characters, operators, and metacharacters that specify patterns to match against input strings. They are concise and easy to understand, making them suitable for describing simple token structures such as identifiers, constants, keywords, and other lexical constructs. Regular expressions can be used in lexical analysis to recognize and tokenize input strings efficiently.

- Context-Free Grammars:** Context-free grammars (CFGs) are used to define the syntactic structure of languages. They consist of a set of production rules that specify how non-terminal symbols can be expanded into a sequence of terminal and non-terminal symbols. CFGs are more powerful than regular expressions and can describe context-free languages, which

Ambiguity

Ambiguity in the context of grammars refers to a situation where a grammar can produce more than one parse tree for a particular input string. This means that there are multiple ways to derive the same string using the production rules of the grammar, leading to different possible interpretations.

include both regular and non-regular languages. They are particularly useful for describing nested structures, such as balanced parentheses, matching begin-end pairs, if-then-else constructs, and other hierarchical language features.

In the given example, the regular expression $(a|b)^*abb$ and the context-free grammar represent the same language, which consists of strings of "a" and "b" that end with "abb". The regular expression provides a concise and straightforward notation for describing the pattern, while the grammar expresses the language in a structured and hierarchical manner.

It's worth noting that regular expressions can describe only regular languages, while context-free grammars can describe both regular and non-regular languages. Regular expressions are typically used in lexical analysis for tokenizing input strings, while grammars are used in syntax analysis to analyze the hierarchical structure of a language.

TOP DOWN PARSING

Top-down parsing is a parsing technique used in computer science and compiler design to construct a parse tree or find a leftmost derivation for an input string. It starts from the top (the start symbol) of a context-free grammar and recursively expands non-terminals to match the input symbols, ultimately producing a parse tree. This parsing technique is also known as predictive parsing because it predicts the production rules to be applied based on the current non-terminal symbol and the next input symbol.

In top-down parsing, the goal is to construct the parse tree in a depth-first, pre-order manner. The process begins with the start symbol as the root of the parse tree and proceeds by expanding non-terminals from left to right, attempting to match the input symbols. The choice of which production rule to apply at each step is determined by the current non-terminal symbol

and the next input symbol. By following this process recursively, the parser explores different paths in the grammar until a successful parse or a parsing error is encountered.

Recursive descent parsing is a common implementation technique for top-down parsing. It involves creating separate recursive procedures or functions for each non-terminal in the grammar. Each procedure handles the parsing logic for its corresponding non-terminal, recursively calling other procedures as necessary to handle the expansion of non-terminals. This approach makes the parsing process intuitive and closely aligns with the grammar's structure.

Top-down parsing has advantages such as simplicity and ease of understanding. It allows for straightforward error reporting and recovery since it detects errors as soon as they occur during the parsing process. However, it can be inefficient in cases where backtracking is required due to ambiguous or non-deterministic grammars. To address this, optimization techniques like memoization or lookahead may be employed to improve the efficiency of top-down parsers.

RECURSIVE DESCENT PARSING

Recursive-descent parsing is a specific type of top-down parsing that uses procedures or functions to handle each non-terminal in the grammar.

Here's a step-by-step explanation of the recursive-descent parsing process:

1. Initialization :-

- Create a parse tree with the start symbol S as the root.
- Set two pointers: one for the parse tree (tree pointer) and one for the input string (input pointer).
- Initially, the tree pointer points to S, and the input pointer points to the first symbol of the input string.

2. Expansion:

- Use the first production rule for the non-terminal pointed by the tree pointer to expand the tree.
- Move the tree pointer to the leftmost symbol of the newly created subtree.

3. Matching:

- Compare the symbol pointed by the tree pointer with the symbol pointed by the input pointer.
- If they match, advance both pointers to the right.
- If they don't match, backtrack to the previous step before the non-terminal expansion and try another production.

4. Recursive Expansion:

- Whenever the tree pointer points to a non-terminal, repeat steps 2 to 4 for that non-terminal.
- Expand the non-terminal using its first production rule.
- Continue matching symbols between the tree pointer and the input pointer.

5. Parsing Completion:

- If the input pointer reaches the end of the input string and the tree pointer passes the last symbol of the tree, parsing is successful.
- Otherwise, if there is no match or no alternative production can be applied, parsing fails.

EXAMPLE :-

Draw a parse tree for the input string "cad" using the following grammar

$S \rightarrow cAd$

$A \rightarrow ab \mid a$

To construct a parse tree for the input string $w = cad$, begin with a tree consisting of a single node labeled S , and the input pointer pointing to c , the first symbol of w . S has only one production, so we use it to expand S and obtain the tree of Fig. 3.2 (a). The leftmost leaf, labeled c , matches the first symbol of input w , so we advance the input pointer to a , the second symbol of w , and consider the next leaf, labeled A .

CONSIDER THE NEXT LEAF, LABELED A .

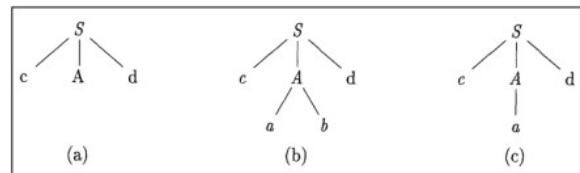


Figure 3.2: Steps in a top-down parse

Now, we expand A using the first alternative $A \rightarrow a b$ to obtain the tree of Fig. 3.2 (b). We have a match for the second input symbol, a , so we advance the input pointer to d , the third input symbol, and compare d against the next leaf, labeled b . Since b does not match d , we report failure and go back to A to see whether there is another alternative for A that has not been tried, but that might produce a match.

In going back to A , we must reset the input pointer to position 2, the position it had when we first came to A , which means that the procedure for A must store the input pointer in a local variable. The second alternative for A produces the tree of Fig. 3.2 (c). The leaf a matches the second symbol of w and the leaf d matches the third symbol. Since we have produced a parse tree for w , we halt and announce successful completion of parsing.

PREDICTIVE PARSING

Predictive parsing is a top-down parsing technique used to construct a parse tree or verify the syntax of a given input string. It is called "predictive" because it predicts which production rule to apply based on the current input symbol and the lookahead symbol. Unlike other top-down parsing methods that use backtracking, predictive parsing eliminates the need for backtracking by using a parsing table that determines the next production to apply.

In predictive parsing, the parser uses a stack to keep track of the grammar symbols and an input buffer to hold the input symbols. The parsing process starts with the stack containing the start symbol of the grammar. The parser then compares the top of the stack with the current input symbol. Based on this comparison, it uses the parsing table to predict the production rule to apply.

The parsing table is typically a two-dimensional table that maps a pair of a non-terminal and an input symbol to a production rule. The non-terminals are the rows of the table, and the input symbols are the columns. Each entry in the table specifies the production rule to use when the given non-terminal and input symbol are encountered.

The predictive parsing algorithm proceeds by repeatedly comparing the top of the stack with the current input symbol. If they match, the parser advances to the next input symbol and pops the stack. If they don't match, the parser consults the parsing table to determine the production rule to apply based on the current non-terminal and the lookahead symbol. The parser then pushes the symbols of the production rule onto the stack in reverse order.

The parsing process continues until the input is completely consumed, and the stack becomes empty. At this point, if the parsing is successful, it means that the input string conforms to the grammar rules, and a valid parse tree can be constructed.

Predictive parsing is efficient because it avoids backtracking and can handle LL(1) grammars, which are a class of grammars where a decision can be made based on one token of lookahead. An LL(1) grammar ensures that the parsing process is deterministic and no conflicts arise in the parsing table.

To implement predictive parsing, a parser generator tool or manual construction of the parsing table can be used. The parsing table is typically generated from the grammar rules and can be constructed by analyzing the first and follow sets of the non-terminals in the grammar.

Overall, predictive parsing is a straightforward and efficient technique for top-down parsing, making it a popular choice for implementing parsers in many programming languages.

LL(1) GRAMMERS

An LL(1) parser is a type of predictive parser that can handle a class of grammars called LL(1) grammars. LL(1) stands for Left-to-right, Leftmost derivation with one symbol of Lookahead. It means that the parser reads the input from left to right and constructs a leftmost derivation of the input string, using one symbol of lookahead to predict the next production rule to apply. "using one symbol of lookahead" means that the LL(1) parser examines the next symbol in the input string to make parsing decisions. In other words, it looks ahead at the next input symbol to determine which production rule to apply.

The LL(1) parsing algorithm is based on a parsing table that determines the next production rule to apply based on the current non-terminal on the stack and the lookahead symbol. The parsing table is typically constructed by analyzing the grammar and computing the first and follow sets of the non-terminals.

The LL(1) parsing algorithm proceeds as follows :-

1. Initialize a stack with the start symbol of the grammar.
2. Read the current input symbol.

3. Repeat the following steps until the stack is empty:
 - a) If the top of the stack is a terminal symbol and matches the current input symbol, pop the stack and read the next input symbol.
 - b) If the top of the stack is a non-terminal symbol, consult the parsing table to determine the production rule to apply based on the current non-terminal and the lookahead symbol.
 - c) If there is a production rule to apply, pop the non-terminal from the stack and push the symbols of the production rule onto the stack in reverse order.
 - d) If there is no production rule to apply, report a syntax error.
4. If the stack is empty and the input symbols have been consumed, the parsing is successful. Otherwise, report a syntax error.

The LL(1) parsing algorithm eliminates the need for backtracking by using the lookahead symbol to make a **deterministic decision at each step**. The LL(1) grammars are carefully designed to avoid conflicts in the parsing table, ensuring that there is a unique production rule to apply for each combination of non-terminal and lookahead symbol.

LL(1) parsers are commonly used in the implementation of programming language compilers and interpreters. They are efficient and can handle a wide range of programming language grammars. However, LL(1) grammars have some restrictions, such as not allowing left recursion, to maintain the determinism and avoid conflicts in the parsing process.

Overall, an LL(1) parser is a type of predictive parser that uses a parsing table and one symbol of lookahead to parse the input string according to the grammar rules. It is a powerful and efficient parsing technique used in many compiler implementations.

BOTTOM – UP PARSING

Bottom-up parsing is a parsing technique that constructs a parse tree for an input string by starting from the leaves (the bottom) and working upwards towards the root (the top). In this approach, the parser attempts to find the rightmost derivation in reverse for the given input string. **It begins with the input string and applies production rules in reverse order to reach the start symbol of the grammar.** If the start symbol can be obtained from the input string, then the string is said to be accepted by the language defined by the grammar.

The process of bottom-up parsing involves repeatedly applying reduction rules to reduce a group of input symbols to a non-terminal symbol until the entire input string is reduced to the start symbol.

A common bottom-up parsing method is called shift-reduce parsing, which involves two main actions:

1. **Shift:** This action involves reading the next input symbol and pushing it onto the stack. The parser keeps shifting symbols from the input buffer to the stack until it can apply a reduction rule.
2. **Reduce:** This action involves applying a production rule in reverse. The parser looks for a sequence of symbols on the top of the stack that matches the right-hand side of a production rule and replaces them with the corresponding non-terminal symbol (the left-hand side of the production). The reduction reduces a group of symbols to a non-terminal.

The shift-reduce parsing continues until the entire input string is reduced to the start symbol of the

grammar. If this is achieved, the parsing process is successful, and the input string is accepted by the language defined by the grammar. Otherwise, if the parser reaches a point where it cannot apply any reduction or shift action, the input string is not valid according to the grammar.

The process of bottom-up parsing can be visualized using a sequence of tree snapshots, where each snapshot shows the reduction steps applied at different stages of parsing. These snapshots illustrate how the input string is gradually reduced to the start symbol of the grammar.

Shift-reduce parsing is a powerful technique that can handle a wide range of context-free grammars, including ambiguous ones. However, it requires careful handling of conflicts that may arise due to multiple possible reductions or shifts at certain points in the parsing process, such as shift-reduce and reduce-reduce conflicts. To handle these conflicts, bottom-up parsers may employ techniques like operator precedence, precedence climbing, or LR (left-to-right, rightmost derivation) parsing methods. LR parsers are a class of bottom-up parsers known for their efficiency and ability to handle a broad class of context-free grammars.

LR PARSING

LR parsing is a bottom-up parsing technique that constructs a parse tree for the input string by performing a series of reductions (reversing the right-hand side of a production) in a rightmost derivation. It stands for "left-to-right, rightmost derivation" and is known for its power and efficiency in parsing a wide range of context-free languages.

In LR parsing, the input string is initially placed on the bottom of a stack. The parser reads the input symbols from left to right and tries to find occurrences of production rules in the reverse order of their right-hand sides. This process involves shifting input symbols onto the stack and reducing them when a production rule is recognized.

The LR parser uses a parsing table, known as the LR parsing table, to determine its actions based on the current state of the parser and the input symbol. The parsing table consists of entries that specify whether to shift the input symbol onto the stack, reduce a portion of the stack using a production rule, or accept the input string as a valid parse.

LR parsing can handle a broader class of grammars than LL parsing, including left-recursive grammars and ambiguous grammars. It is commonly used in industrial-strength compiler generators and parser generators due to its efficiency and ability to handle complex languages.

There are different types of LR parsers, such as LR(0), SLR(1), LALR(1), and LR(1), each with varying degrees of lookahead and parsing power. These types differ in terms of the information they use to make parsing decisions and the size of their parsing tables.

In summary, LR parsing is a bottom-up parsing technique that constructs a parse tree by performing reductions based on the rightmost derivation of the input string. It uses a parsing table to guide its actions and can handle a wide range of context-free languages efficiently.

CHAPTER FIVE

SYMBOL TABLE , SYNTAX RELATED TRANSLATION AND TYPE CHECKING

A symbol table is a data structure used by compilers to store and manage information about identifiers (such as variables, functions, classes) encountered in the source code of a program. It acts as a central repository for collecting and organizing information about these program constructs.

The symbol table is built incrementally during the analysis phases of a compiler and utilized during the synthesis phases for generating the target code. Each entry in the symbol table contains

relevant information about an identifier, such as its name (lexeme), data type, memory location, scope, and other properties specific to the programming language.

The symbol table plays a crucial role in maintaining scope and binding information. It helps in resolving references to identifiers and ensures that they are used correctly and consistently within the program. When a name is encountered in the source code, the symbol table is searched to retrieve information about that name. If a new identifier is discovered or additional information about an existing identifier is found, the symbol table is updated accordingly.

The primary purposes of a symbol table are :-

1. **Storing entity names:** It provides a structured form to store the names of all entities (variables, functions, classes, etc.) encountered in the program. This allows for efficient access and retrieval of information.
2. **Declaration verification:** The symbol table is used to check if a variable or any other entity has been properly declared before its usage. It helps in ensuring that the program follows the language's scoping and declaration rules.
3. **Type checking:** The symbol table is involved in the process of type checking, which ensures that assignments and expressions in the source code are semantically correct. It verifies that the operations performed on variables or expressions are compatible with their declared types.
4. **Scope resolution:** The symbol table helps determine the scope of a name, i.e., where the name is valid and accessible within the program. It assists in resolving conflicts or ambiguities that may arise due to the presence of the same name in different scopes.

Efficient symbol table mechanisms are essential to support fast insertion and retrieval of entries.

Depending on the size and complexity of the language being compiled, symbol tables may be implemented using different data structures such as hash tables, binary search trees, or other suitable data structures to ensure efficient symbol management during the compilation process.

INTRODUCTION TO TYPE CHECKING

Type checking is a crucial process performed by a compiler to ensure that the source program adheres to the syntactic and semantic rules of the programming language. It is a static checking process that occurs during compilation, prior to the actual execution of the program.

The main objective of type checking is to detect and prevent type errors, which occur when an operation or expression is applied to incompatible data types. Type errors can lead to unexpected behavior or runtime errors in the compiled program. By performing type checks, the compiler ensures that the program operates with consistent and compatible data types, promoting safer and more reliable code execution.

Static type checking involves examining the program's structure, expressions, and statements to verify their compatibility and correctness according to the language's type system. Here are some key aspects of type checking :-

1. **Type Compatibility:** The type checker verifies that the operands and operators used in expressions or statements are compatible. For example, adding an integer and a string would be flagged as a type error.
2. **Type Inference:** In languages with type inference, the type checker deduces the types of variables or expressions based on their usage and context. This allows for implicit type declarations without explicit type annotations.
3. **Type Consistency:** The type checker ensures that the assigned value or result of an expression is consistent with the declared or expected type. For instance,

assigning a floating-point value to an integer variable may require a type conversion or trigger a type error.

4. **Function and Procedure Calls:** The type checker validates that function and procedure calls match the expected number, order, and types of arguments. It checks that the arguments provided correspond to the defined parameter types.
5. **Data Structures and Operations :-** Type checking ensures that operations such as indexing, dereferencing, or member access are applied to compatible data structures. For example, accessing an element in an array requires an integer index, and accessing a field in a struct requires the correct field name.
6. **Type Safety:** Type checking promotes type safety by preventing implicit type conversions that may lead to data loss or unpredictable behavior. It enforces explicit type conversions or casting when necessary.

The type checker analyzes the program's structure, symbol table entries, and type declarations to perform these checks. If any type errors are detected, the compiler reports them as compilation errors or warnings, allowing the programmer to address and fix them before executing the program.

Overall, type checking plays a vital role in ensuring the integrity and correctness of the program's type system, promoting robustness and reliability in the compiled code.

TYPE SYSTEMS

A type system is a fundamental component of programming languages that governs the classification and organization of data in a program. It provides a set of rules and mechanisms for assigning types to various language constructs, such as variables, expressions, functions, and data structures.

In the context of type checking, the design of a type checker relies on information about the syntactic constructs in the language, the concept of types, and the rules for assigning types to different language constructs. The type system defines the properties and behavior of these types and how they interact with each other.

In many programming languages, including Pascal and C, types are classified as either basic or constructed.

1. **Basic Types:** Basic types are atomic types that do not have any internal structure as far as the programmer is concerned. They represent fundamental data types supported by the language. Examples of basic types in Pascal include boolean, character, integer, and real. Additionally, Pascal allows the construction of other basic types, such as enumerated types (e.g., (violet, indigo, blue, green, yellow, orange, red)) and subrange types (e.g., 1...10). Basic types provide the building blocks for constructing more complex types.
2. **Constructed Types:** Constructed types are formed by combining basic types and other constructed types. They have a composite structure and are used to represent more complex data structures. Common examples of constructed types are arrays, records (structs in C), and sets. Arrays are collections of elements of the same type, while records consist of multiple fields or members, each with its own type. Sets represent a collection of distinct values from a predefined range.

The type system also defines rules for type compatibility and type coercion, specifying how types can be used and manipulated in expressions and operations. For example, the type system may dictate that addition, subtraction, and multiplication operators can only be applied to operands of the same basic type (such as integers), and the result will also be of the same type.

Furthermore, the type system allows for the construction of derived types, such as pointers in C, which are used to store memory addresses of other objects. Pointers have a type associated with them, often expressed as "pointer to type," indicating the type of the object they point to. The type system ensures that operations involving pointers adhere to the correct type rules.

By enforcing type rules and performing type checking, the type system helps prevent type errors and ensures that programs follow the correct usage of types, leading to safer and more reliable code execution.

Compiler writers rely on the information provided by the language specification, such as the Pascal report or the C reference manual, to define the type system and implement the type checker accordingly. The type system forms a foundation for the compilation process, guiding the analysis and synthesis phases of the compiler to generate correct and efficient target code.

SYNTAX RELATED TRANSLATION